StudentIDs:	293867	Score:
Class:	Data Warehouse Mini-Project	
Date:	17-05-2026	

This mini project assignment consists of 3 tasks. You should focus on providing complete solution, however if you are unable to solve a particular step, try to give at least a partial solution or provide justification for the reason for the lack of a solution.

MINI-PROJECT 2 – DESIGN

The next step of creating a dimensional data warehouse is the design of the dimensional model. Having specified the "business needs" and reality (problem field and data details) of the available data resources, we can start designing an adequate dimensional solution. In short, you should already understand the domain, know all the facts and contexts of their analysis (from stage 1). Further, using the dimensional model you can specify the blueprint for the ETL process. You need to specify the required data movement tasks (in the form of logical data map) and the plan of data handling (in the form of a basic ETL map).

The process of implementing an ETL system for a data warehouse system should be preceded by a deep and thorough understanding of the "business needs" and reality (problem field). A particular implementation of data movement and integration tasks can end up in a disaster if the goal is ill-defined or unclear. As such, here we focus on the analysis part and introduce a proper multidimensional model, i.e., model capable of handling all the identified user information needs.

TASK 1 – FINAL GOAL – DETAILS:

Please state the identified main business process being the focus of the project. Identify main decision problem(s) associated with it. Provide results in this section 1.0.1.

For the identified main business process, please identify basic users (types) – provide brief description of each type – and state expectations of each type. Do not select too many types – as it might be beneficial to focus on more details for 2-3 user types. Further present detailed needs by specify 10 selected OLAP user query types (do not write any SQL queries, these should be formulated in natural language – strictly in business terms). Note – these should not be very specific queries (using specific attribute values), but rather general query/need types. For each query/need please indicate how a user can utilise resultant information. Match each query to the identified user types. Provide results in section 1.0.2.

Sample inquiries:

- *How have total sales of our products evolved over the past five years, by region and sales channel?*
    - *Usage: The analysis allows you to identify the regions and channels that are most profitable or need support. Based on this, decisions can be made to increase marketing investments in high-potential regions or restructure operations in less profitable areas.*
- *Which products generated the most profit in the current quarter, and which in the same period last year?*
    - *Usage: This allows you to quickly capture changes in customer preferences and sales effectiveness. Decision makers can decide to expand the range of best-selling products or withdraw those that are loss-making.*
- *Which customers make the most purchases and what is their demographic structure (e.g. age, gender, location)?*
    - *Usage: Allows you to create personalized offers and loyalty campaigns, increasing the chances of retaining key customers and increasing the value of their shopping cart.*

Perform basic analysis of these query types (1.0.2), try to infer some general user requirements. Focus on identifying event(s) and perspectives that the user is interested in. Further, perform basic analysis of user query types (1.0.2), try to infer some detailed user requirements. List all required and possible measures and individual dimensions. Provide results in section 1.0.3. Remember to define the grain (the level of detail for a single row in the fact table and dimensions). In section 1.0.4 provide a Dimensional Bus Matrix based on your findings in section 1.0.3. Remember that Dimensional Bus Matrix is a grid where rows are different identified Business Processes (facts) and columns are different identified dimensions. It shows which dimensions are shared (conformed) across different facts. Finally, in section 1.0.5, comment on consistency and evaluate your proposal (1.0.1-1.0.4).

1.0.1 THE FOCUS OF THE PROJECT

The business process examined is Bitcoin Transaction Settlement. Every confirmed on-chain transaction is a settled economic event, triggered every block validation block (~10 minutes). The main decision problems revolve around transaction cost optimization, speed of settlement (fee rates), protocol upgrades monitoring (SegWit vs. Legacy script types), and macro market-sentiment transaction correlations.

1.0.2 EXPECTATIONS AND DETAILED NEEDS FOR DECISION SUPPORT

Primary User Profile: **Blockchain Analyst / Researcher**. Centered entirely on on-chain network transaction settlement dynamics, fee dynamics, script adoption, and macro-financial correlations.

Streamlined User Needs (Q1-Q8):

Q1: SegWit adoption rates. Tracks upgrade curves of network protocol.

Q2: Network congestion & fee dynamics. Informs wallet developers and spenders on fee volatility.

Q3: Fee burden as % of settled USD output. Critical for modeling on-chain transfer microeconomics.

Q4: Market sentiment (Fear & Greed Index) vs. transaction settlement metrics. Identifies momentum shifts.

Q5: Multi-input/multi-output spending composition. Helps privacy researchers evaluate UTXO clustering.

Q6: Transaction counts and sizes across block height halving eras. Evaluates scalability upgrades.

Q7: Coinbase payout transaction volumes vs. standard transfers. Models miner payout patterns.

Q8: Network Value to Transactions (NVT Ratio) behavior during neutral vs extreme market days. Detects over/undervaluations.

1.0.3 SCOPE OF ANALYSIS – ASPECTS EXAMINED

Event: On-chain transaction settlement. Perspective (Dimensions): Date, Block, Transaction Type, Market daily indicators. Grain: One confirmed on-chain transaction. Measures include satoshi-level metrics (fee_satoshis, input_value_sat, output_value_sat), sizes (tx_size_bytes, tx_weight_units), and daily conformed market values (btc_price_usd_avg, input_value_usd, output_value_usd, fee_usd, market_cap_usd, fear_greed_score).

1.0.4 BUS MATRIX

Include a Dimensional Bus Matrix. This is a grid where rows are different identified Business Processes (facts) and columns are different identified Dimensions. It shows which dimensions are shared (conformed) across different facts.

Business Process	DIM_DATE	DIM_BLOCK	DIM_TX_TYPE	DIM_MARKET
Bitcoin Transaction Settlement (FACT_TRANSACTION)	✓	✓	✓	✓

The Dimensional Bus Matrix establishes our conformed data architecture. It shows how our single business process (Bitcoin Transaction Settlement) shares standard conformed dimensions (DIM_DATE, DIM_BLOCK, DIM_TX_TYPE, DIM_MARKET) across the entire platform, ensuring data consistency.

1.0.5 CONSISTENCY EVALUATION

Double-check the consistency and completeness of your previous findings 1.0.1-1.0.4.

The dimensional model is highly consistent. The fact table grain (one confirmed transaction) is conformed to every dimension. All queries (Q1-Q8) are completely answered utilizing the conformed dimensions. The daily average price is conformed to date keys, which provides deterministic input/output USD calculations without breaking the business grain.

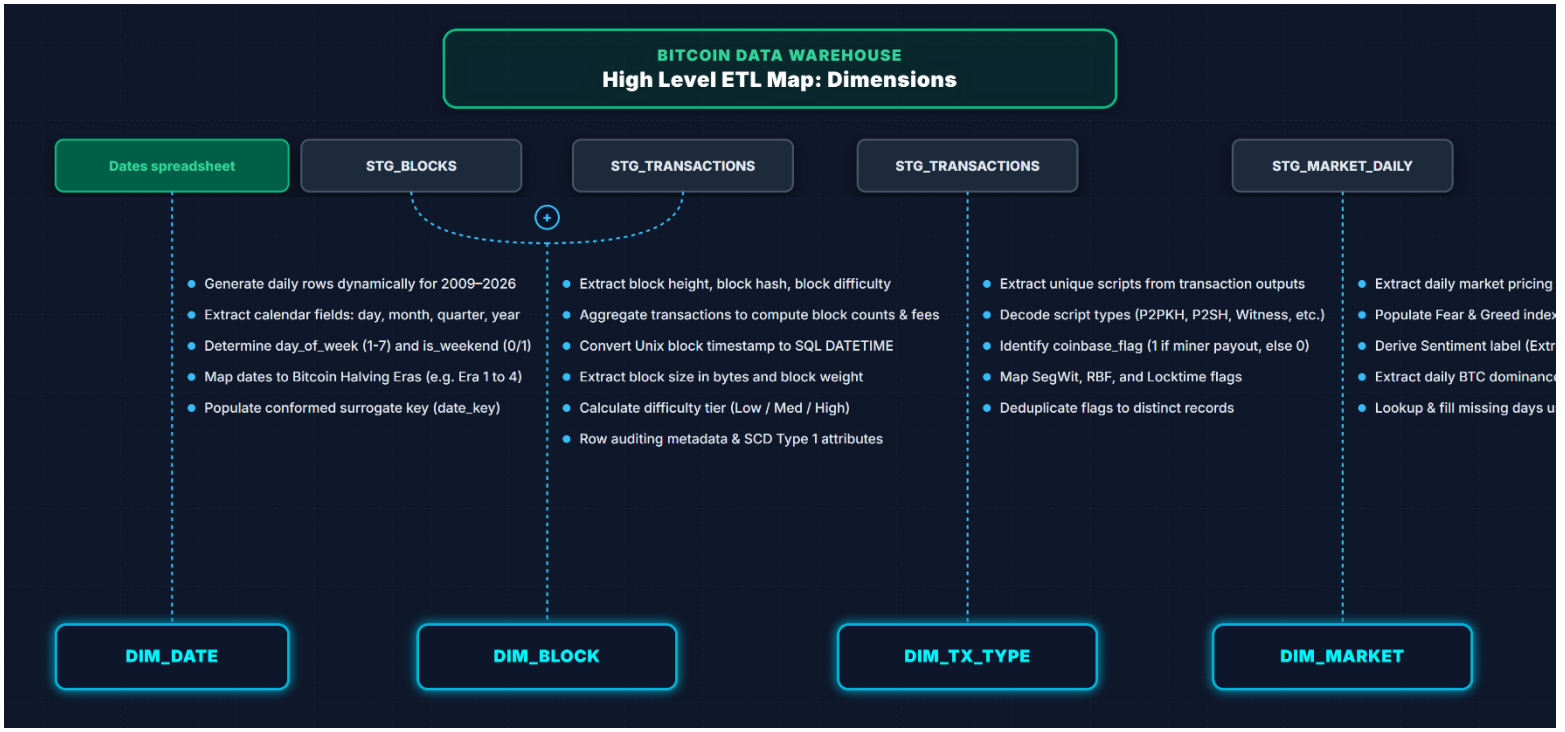
TASK 2.1 – HIGH-LEVEL DIMENSIONAL MODEL

Please prepare the dimensional model for the selected dataset and specified user requirements, in accordance with the specification below and discuss the solution with the lecturer.

2.1.1 HIGH-LEVEL DIMENSIONAL MODEL

Please, define a high-level dimensional schema and prepare a high-level diagram for the intended multi-dimensional model. First, briefly present and explain your design solution for the fact table. Next, focus on presentation and explanation of high-level dimensions.

We design a conformed Star Schema centered around FACT_TRANSACTION. The fact table holds transaction measures and foreign keys to DIM_DATE (daily grain), DIM_BLOCK (block grain), DIM_TX_TYPE (flag grain), and DIM_MARKET (daily macro snapshot grain). This star schema supports high-performance querying and is highly intuitive for OLAP analysis.

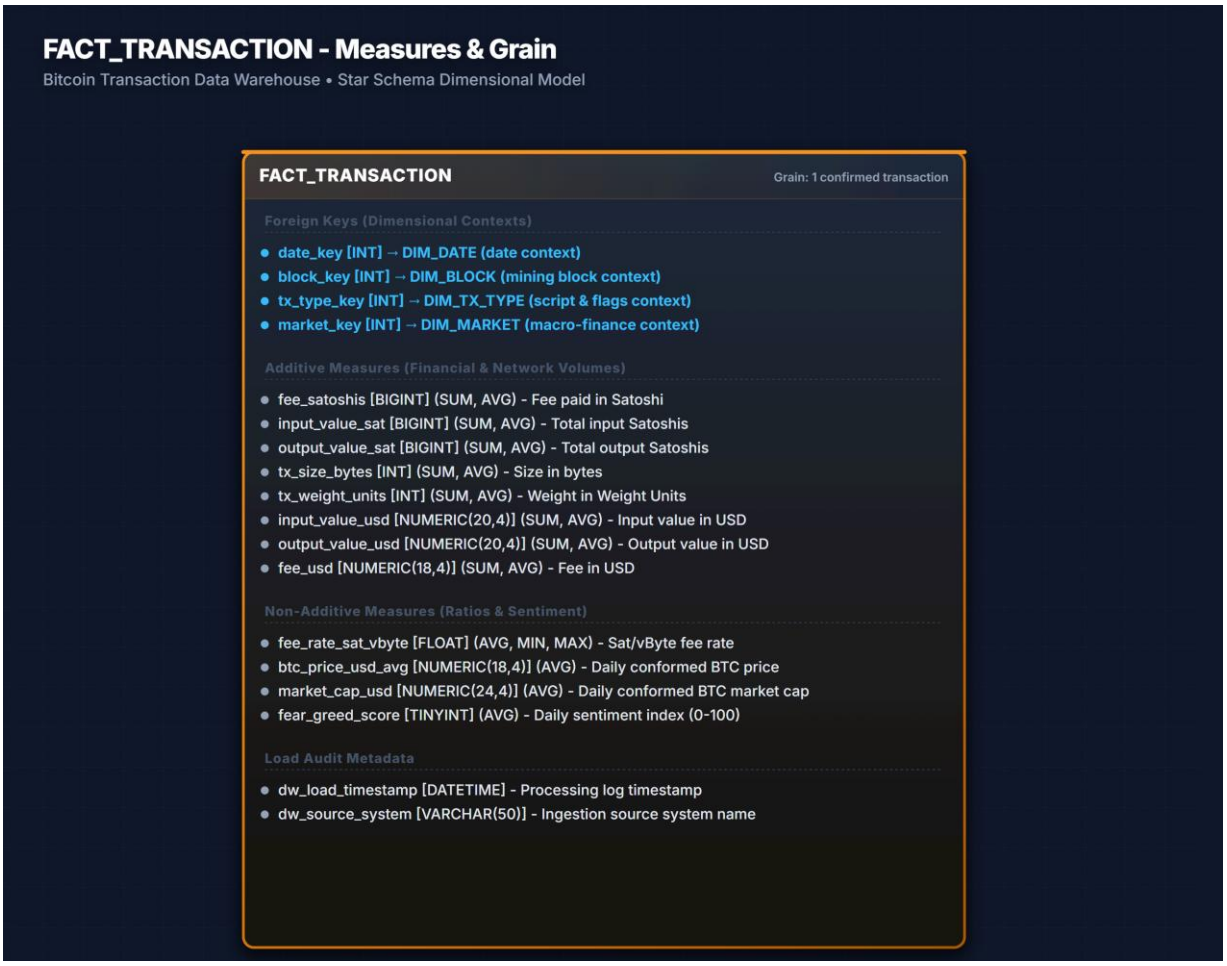


TASK 2.2 – DETAILED DIMENSIONAL MODEL

Based on 2.1.1 start with highlighting the general structure, i.e., name facts and measures, name dimensions, and list required individual dimensions (dimension's attributes). Next provide details of the schema.

2.2.1 FACT(S) AND MEASURES

	Fact	Measure(s)	Type	Aggregation	Grain
1	FACT_TRANSACTION	fee_satoshis	Additive	SUM, AVG	1 transaction
2	FACT_TRANSACTION	fee_rate_sat_vbyte	Non-additive	AVG, MIN, MAX	1 transaction
3	FACT_TRANSACTION	input_value_sat	Additive	SUM, AVG	1 transaction
4	FACT_TRANSACTION	output_value_sat	Additive	SUM, AVG	1 transaction
5	FACT_TRANSACTION	tx_size_bytes	Additive	AVG, SUM	1 transaction
6	FACT_TRANSACTION	tx_weight_units	Additive	AVG, SUM	1 transaction
7	FACT_TRANSACTION	btc_price_usd_avg	Non-additive	AVG	1 transaction (daily conformed)
8	FACT_TRANSACTION	input_value_usd	Additive	SUM, AVG	1 transaction
9	FACT_TRANSACTION	output_value_usd	Additive	SUM, AVG	1 transaction
10	FACT_TRANSACTION	fee_usd	Additive	SUM, AVG	1 transaction
11	FACT_TRANSACTION	market_cap_usd	Non-additive	AVG	1 transaction (daily conformed)
12	FACT_TRANSACTION	fear_greed_score	Non-additive	AVG	1 transaction (daily conformed)



2.2.2 CONTEXT FOR FACTS

	Dimension Table	Type	Grain	Description
1	DIM_DATE	Standard	1 calendar day	Dates and halving cycle metrics
2	DIM_BLOCK	Standard	1 mined block	Block properties (hash, height, difficulty, size)
3	DIM_TX_TYPE	Standard	Unique flag combination	Transaction script type and transaction flags (SegWit, coinbase)
4	DIM_MARKET	Standard	1 unique market snapshot	Daily market metrics (market cap, Fear & Greed index)

2.2.3 CONTEXT DETAILS

	Dimension Table	Attribute	Type	SCD	Description
1	DIM_DATE	date_key	INT	Type 0	Surrogate Date Key (YYYYMMDD)
2	DIM_DATE	date	DATE	Type 0	Calendar date
3	DIM_DATE	day	TINYINT	Type 0	Day of the month
4	DIM_DATE	month	TINYINT	Type 0	Month index (1-12)
5	DIM_DATE	quarter	TINYINT	Type 0	Calendar quarter
6	DIM_DATE	year	INT	Type 0	Calendar year
7	DIM_DATE	day_of_week	TINYINT	Type 0	Day index (1=Sun, 7=Sat)
8	DIM_DATE	is_weekend	TINYINT	Type 0	Weekend flag (1/0)
9	DIM_DATE	halving_era	VARCHAR(20)	Type 0	Halving epoch (e.g. Era 4)
10	DIM_BLOCK	block_key	INT	Type 0	Surrogate Block Key
11	DIM_BLOCK	block_height	INT	Type 0	Block height (business key)
12	DIM_BLOCK	block_hash	VARCHAR(64)	Type 0	SHA-256 block hash
13	DIM_BLOCK	block_timestamp	DATETIME	Type 0	Mining timestamp
14	DIM_BLOCK	block_size_bytes	INT	Type 0	Raw size in bytes
15	DIM_BLOCK	block_weight_units	INT	Type 0	Weight units
16	DIM_BLOCK	block_difficulty	FLOAT	Type 0	Proof-of-work difficulty target
17	DIM_BLOCK	difficulty_tier	VARCHAR(20)	Type 0	Tier (Low/Medium/High/Extreme)
18	DIM_TX_TYPE	tx_type_key	INT	Type 1	Surrogate Transaction Type Key
19	DIM_TX_TYPE	script_type_desc	VARCHAR(20)	Type 1	Dominant script (P2PKH, P2TR, etc)
20	DIM_TX_TYPE	segwit_flag	TINYINT	Type 1	SegWit indicator (1/0)
21	DIM_TX_TYPE	coinbase_flag	TINYINT	Type 1	Coinbase transaction flag (1/0)
22	DIM_TX_TYPE	rbf_flag	TINYINT	Type 1	Replace-By-Fee flag (1/0)
23	DIM_TX_TYPE	locktime_flag	TINYINT	Type 1	Locktime flag (1/0)

24	DIM_MARKET	market_key	INT	Type 1	Surrogate Market Key
25	DIM_MARKET	fear_greed_score	TINYINT	Type 1	Daily sentiment score
26	DIM_MARKET	fear_greed_label	VARCHAR(20)	Type 1	Sentiment label (Fear, Greed)
27	DIM_MARKET	btc_dominance_percent	FLOAT	Type 1	Bitcoin dominance percentage
28	DIM_MARKET	market_cap_usd	NUMERIC(24,4)	Type 1	Daily circulating market cap
29	DIM_MARKET	volatility_index	FLOAT	Type 1	Daily volatility score

*Remember to define strategy for **Slowly Changing Dimensions (SCD)**. You must define how to handle updates (e.g., does a customer's address change overwrite the old one, or do we keep a history?).*

2.2.4 DIMENSIONAL SCHEMA – DATABASE DIAGRAM

Please, define schema and prepare a diagram for multi-dimensional model (star, snowflake, or fact constellation). First, briefly present and explain your design solution for the fact table. Next, focus on presentation and explanation of dimension tables. Finally, design and present the final dimensional diagram.

The Database Diagram represents a clean Star Schema. FACT_TRANSACTION is placed at the center, surrounded by conformed dimensions. The foreign key connections are defined on date_key, block_key, tx_type_key, and market_key. Every table includes auditing load columns: dw_load_timestamp (datetime) and dw_source_system (varchar) to ensure full traceability and data lineage.

Recommendation: Every table should include columns like dw_load_timestamp and dw_source_system. This allows users to know exactly when the data was processed and where it came from.

TASK 3.1 – ETL MODEL – DETAILS:

Please prepare the model of the ETL process, in accordance with the below specification and discuss the solution with the lecturer.

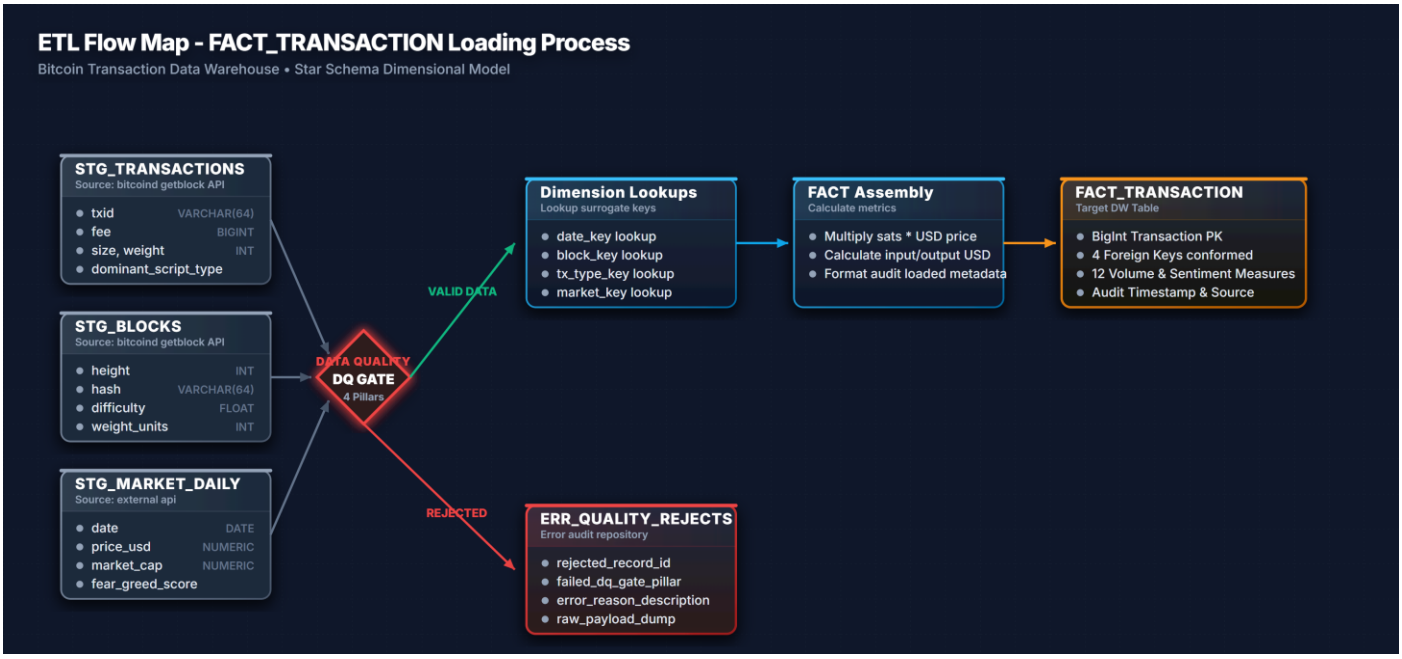
TASK 3.2 – ETL MODEL – SOLUTIONS:

3.2.1 ETL MAP

Prepare an ETL map that specifies the required operations on data in the ETL process. Draft the map at a relatively high level of detail (focusing on dimension tables, rather than individual source-to-target mappings, as in logical data map), use a box to represent each utilised source table (file, spreadsheet, or other source), and a box to represent each dimensional table (table in the dimensional model). In short, the map should specify all operations, which are required to obtain the final dimensional table. Please, specify a map separately for each dimension table and separately for each fact table. Exemplar map is provided at the end of this document – in your submission, please remove them. This "ETL Map" is regarded as a high-level visual representation of a detailed mapping found in Logical Data Maps. We are using this simplified approach, as in a real-world scenario the source-to-target field mapping is often the most time-consuming document to create.

NOTE: basic example of an ETL map is provided at the end of this document.

*When drafting the map for each table, insert **"Quality Gates"** between the Source and stages. Add a specific box or symbol (like a diamond) labeled "DQ Gate" to your diagram. If a record fails the gate, it should be routed to an "Error/Reject Table" rather than the Target.*



3.2.2 LOGICAL DATA MAP FOR A SINGLE DIMENSION

For a single selected (you can pick a dimension you prefer) dimension create a detailed logical data map.

	Target table	*Target attribute*	*Target data type*	*Source system/table*	*Source attribute*	*Transformation logic / rule*
1	*DIM_TX_TYPE*	*tx_type_key*	*INT*	*ETL Engine*	*-*	*Surrogate Key, generated via IDENTITY(1,1)*
2	*DIM_TX_TYPE*	*script_type_desc*	*VARCHAR(20)*	*STG_TRANSACTIONS*	*dominant_script_type*	*If NULL, map to 'OTHER'. Trim whitespace and cast upper.*
3	*DIM_TX_TYPE*	*segwit_flag*	*TINYINT*	*STG_TRANSACTIONS*	*is_segwit*	*Convert boolean to TINYINT: CASE WHEN is_segwit=1 THEN 1 ELSE 0 END*

4	*DIM_TX_TYPE*	*coinbase_flag*	*TINYINT*	*STG_TRANSACTIONS*	*is_coinbase*	*Convert boolean to TINYINT: CASE WHEN is_coinbase=1 THEN 1 ELSE 0 END*
5	*DIM_TX_TYPE*	*rbf_flag*	*TINYINT*	*STG_TRANSACTIONS*	*rbf_enabled*	*Convert boolean to TINYINT: CASE WHEN rbf_enabled=1 THEN 1 ELSE 0 END*
6	*DIM_TX_TYPE*	*locktime_flag*	*TINYINT*	*STG_TRANSACTIONS*	*locktime*	*Flag locktime: CASE WHEN locktime > 0 THEN 1 ELSE 0 END*
7	*DIM_TX_TYPE*	*dw_load_timestamp*	*DATETIME*	*ETL Engine*	*-*	*Insert load timestamp: GETDATE()*
8	*DIM_TX_TYPE*	*dw_source_system*	*VARCHAR(50)*	*ETL Engine*	*-*	*Hardcoded audit identifier: 'STG_TRANSACTIONS'*

As transformation use a general information what needs to be done, e.g., "convert Date to YYYYMMDD integer format", "concatenate: FirstName + ' ' + LastName", "create Surrogate Key".

3.2.3 DATA QUALITY RULES & ACTIONS

*Describe your **Data Quality (DQ)**. Modern ETL processes require "Quality Gates" or "Expectations" to be defined within the flow to prevent "garbage in, garbage out". Specify what happens when data is "bad." Example: If a Store_ID in the source doesn't exist in the Dim_Store table, do you reject the record, or assign it to a "Unknown" member?*

	Dimension/Fact	*Attribute*	*DQ Rule*	*Action*
1	*FACT_TRANSACTION*	*txid*	*Must be unique and follow a 64-char hex format.*	*Reject record, insert into ERR_QUALITY_REJECTS, log warning.*
2	*FACT_TRANSACTION*	*output_value_sat*	*Value cannot be NULL or negative (must be >= 0).*	*Reject record, insert into ERR_QUALITY_REJECTS.*
3	*DIM_TX_TYPE*	*script_type_desc*	*Must match: 'P2PKH', 'P2SH', 'P2WPKH', 'P2WSH', 'P2TR', 'OTHER'.*	*Map to 'OTHER', log discrepancy, continue loading.*
4	*DIM_MARKET*	*date*	*Date must represent a snapshot from the last 24 hours.*	*Send alert to operations team; insert default values.*
5	*DIM_BLOCK*	*block_hash*	*Cannot be NULL and must be exactly 64 characters long.*	*Reject block record and all child transactions in load batch.*

Make sure to justify your ETL process, explicitly focusing on these four modern DQ pillars:

1. ***Completeness:** Are there mandatory fields (like TotalAmount) that are coming in as NULL?*
2. ***Uniqueness:** How are you ensuring no duplicate records are loaded into the target?*
3. ***Consistency:** Does the "Gender" code M/F consistently decode to Male/Female across all sources?*
4. ***Freshness:** Does the data reflect the current business state? (e.g., was the source file updated within the last 24 hours?) .*

GENERAL CONCLUSIONS:

Use this section to provide your general conclusions:

The successful design and modeling of the Bitcoin Transaction Data Warehouse marks a critical step towards enabling high-performance, multi-dimensional analysis of blockchain activity. By adopting a formal Star Schema approach, we have effectively transformed deeply nested and complex operational data (extracted directly from the `bitcoind getblock` API) into an analytical format optimized for query performance and business intelligence.

At the core of this architecture is the `FACT_TRANSACTION` table, capturing data at the lowest, most atomic grain: a single confirmed transaction. This allows for unparalleled flexibility in aggregating and drilling down into critical metrics such as fee economics (sat/vByte), network volume (satoshis moved), and block weight utilization.

The supporting dimensions—`DIM_BLOCK`, `DIM_TX_TYPE`, `DIM_DATE`, and `DIM_MARKET`—provide the necessary context to slice the transactional facts. Crucially, by integrating external market data (e.g., USD price and Fear & Greed indices), the model transcends pure network analytics to offer deep financial and behavioral insights into the Bitcoin ecosystem.

Furthermore, the defined ETL pipeline ensures analytical integrity. By explicitly flattening the complex API responses into discrete staging tables (`STG_TRANSACTIONS` and `STG_BLOCKS`) and enforcing strict Data Quality Gates, we guarantee that only valid, conformed data enters the presentation layer.

In conclusion, this Data Warehouse provides a robust, scalable foundation. It not only addresses the immediate analytical requirements regarding transaction throughput and miner economics but is also highly extensible, allowing for the future integration of new data sources (such as Mempool statistics or Lightning Network states) without disrupting the existing dimensional model.